

## Intermediate Test — Python Advanced

|                        |                     |                        |
|------------------------|---------------------|------------------------|
| <b>Total Marks: 30</b> | <b>Time: 1 Hour</b> | <b>Date: June 2025</b> |
| Name: _____            | Roll No: _____      | Batch: _____           |

### Section 1 — OOP (10 Marks)

**Q1. Create a class `Employee` with: (3 Marks)**

- Private variable `__salary = 50000`
  - Method `increment()` → `salary += 10000`
  - Method `deduct()` → `salary -= 5000`
  - Method `get_salary()` → print salary
- Create 2 objects and call all methods on both.

**Q2. Create Abstract class `Vehicle` with: (4 Marks)**

- Abstract methods: `start()`, `stop()`, `fuel_type()`
- Create 3 child classes:
- Car → 'Car started' / 'Petrol'
  - Bike → 'Bike started' / 'Petrol'
  - Tesla → 'Tesla started' / 'Electric'
- Create objects and call all methods.

**Q3. Create a Hybrid Inheritance program: (3 Marks)**

- Father: `property()`, `business()`
  - Son(Father): `study()`
  - Daughter(Father): `dance()`
  - GrandChild(Son, Daughter): `gaming()`
- Create object of GrandChild and call ALL methods.

### Section 2 — Exception Handling (8 Marks)

**Q4. Create class `AgeVerification`: (4 Marks)**

- Method `set_age(age)`:
    - If `age < 0` : raise `ValueError`
    - If `age < 18` : raise custom `UnderAgeError`
    - If `age > 100` : raise custom `InvalidAgeError`
    - Else : print 'Valid age!'
- Handle all exceptions with finally block.

**Q5. Create a `Login System`: (4 Marks)**

- Private variable `__password = 'python@123'`
- Private variable `__attempts = 3`
- Method `login(password)`:
  - Wrong password: `attempts -= 1`, show remaining
  - If `attempts == 0`: raise `AccountLockedError`

→ Correct: print 'Login successful!'  
Handle AccountLockedError with finally block.

### Section 3 — File Handling (6 Marks)

**Q6. Create a Student Report System: (3 Marks)**

- Create 'report.txt' with 5 students and marks:  
Rahul-85, Priya-90, Rohan-78, Sneha-92, Amit-65
- Read file and print only students with marks > 80
- Handle FileNotFoundError with finally block.

**Q7. Create an Employee File System: (3 Marks)**

- Create 'employees.txt' with 3 employees
- Read the file and print
- Append 2 more employees
- Read updated file
- Delete the file and verify using os.path.exists()

### Section 4 — JSON and API (6 Marks)

**Q8. Create a Student Database using JSON: (3 Marks)**

- Create list of 3 students (name, age, city, marks)
- Save to 'students.json'
- Read the file and print only students with marks > 75
- Handle FileNotFoundError

**Q9. Call this API and filter results: (3 Marks)**

- URL: <https://jsonplaceholder.typicode.com/users>
- Print details of users from city 'Gwenborough' only
  - For each user print: Name, Email, Phone, Company name
  - Check status code == 200 before processing
  - Handle all exceptions properly

#### Marking Scheme

| Section   | Topic              | Marks     |
|-----------|--------------------|-----------|
| Section 1 | OOP                | 10        |
| Section 2 | Exception Handling | 8         |
| Section 3 | File Handling      | 6         |
| Section 4 | JSON & API         | 6         |
|           | <b>Total</b>       | <b>30</b> |

| Marks   | Grade            |
|---------|------------------|
| 27 - 30 | Excellent ★★★    |
| 21 - 26 | Good ★★          |
| 15 - 20 | Average ★        |
| 0 - 14  | Need Improvement |

---

All the best! ■